

EXERCISE 02 · UNIT 1

Cache blocking — make it faster without threads

The same $2n^2$ memory operations, reordered. No threads, no GPU, no cleverer algorithm, and roughly twice the speed.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

EXERCISE

ex02-cache-blocking

LANGUAGE

C, single-threaded

SUBMIT AS

<ROLLNO>.c

UNIT

1 — Architecture and performance

MARKS

12

OFFERING

Odd Semester 2026-27

The problem

Transpose an $n \times n$ matrix: `b[j*n + i] = a[i*n + j]`.

The naive version is two lines and correct. It is also slow, for a reason you can state exactly.

Why the naive version is slow

```
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        b[j*n + i] = a[i*n + j];
```

The **read** walks a row: consecutive iterations touch consecutive addresses, so each 64-byte cache line delivers eight useful doubles.

The **write** walks a column: consecutive iterations are n doubles apart. At $n = 4096$ that is a 32 KB stride, so every single write touches a fresh cache line, uses 8 bytes of it, and the line is evicted long before the loop comes back for its neighbours.

DEFINITION · THE ESSENTIAL ASYMMETRY OF TRANSPOSITION

One of the two accesses **must** stride, because transposing inverts the index order. You cannot make both contiguous by swapping the loops: that just moves the problem from the write to the read.

The fix is not to make the stride disappear but to make the **working set small enough that the strided accesses still hit in cache**.

Blocking

Transpose one square block at a time instead of one element at a time:

```
for each block (ii, jj):
    for i in that block:
        for j in that block:
            b[j*n + i] = a[i*n + j];
```

Now both `a` and `b` are confined to a $b \times b$ region while it is being worked on. Two such blocks must fit in cache together, so $2 \cdot b^2 \cdot 8$ bytes should be comfortably smaller than your L2.

Block sizes from 8 to 64 are worth trying. **Find the best one by measuring**, and say in a comment which value you chose and why. That measurement is the exercise.

PITFALL · N IS NOT ALWAYS A MULTIPLE OF YOUR BLOCK SIZE

The last row and column of blocks are ragged. Clamp with `min(ii + BS, n)` rather than assuming the division is exact. The hidden tests use $n = 3, 31, 63, 127$ and 257 precisely to catch this, and one of them checks that you did not write past element $n^2 - 1$.

Single-threaded, deliberately

No OpenMP. No pthreads. The grader rejects both.

BEYOND THE SYLLABUS · WHY THE RULE EXISTS

You will be tempted to reach for `#pragma omp parallel for`. The point of this exercise is that **memory behaviour is worth a large factor before you add a single thread**, and that parallelising a cache-hostile loop just gives you many cores all waiting on the same overloaded memory system.

In the Unit 2 matrix-multiply worked example the serial fixes were worth 13.5x on their own, before any threads were created. The two multiply together, and the serial one comes first.

Building and testing

```
./selfcheck.sh
```

To find your best block size, time it yourself at several values before submitting. On the cluster:

```
srun --cpus-per-task=4 --time=00:10:00 ./bench 4096 6
```

How this is marked

CHECK	MARKS	WHAT IT MEANS
Compiles cleanly	1	Warnings as errors
Single-threaded only	1	No OpenMP, no pthreads
Public tests	3	Correctness, including ragged sizes
Hidden tests	3	Primes, small n , and a write-overflow check
Beats the naive transpose	4	At least 1.35x at $n = 4096$, on one core

The speedup threshold is set from measurement: a correctly blocked version measures about 1.7x on the reference machine, so 1.35x leaves room for a different cache hierarchy without letting an unblocked answer through.

Submitting

AIE23001.c